Plano de Trabalho em Dupla - A-Maze-ing

Organizacao para duas pessoas codarem ao mesmo tempo, em paralelo, sem ficarem presas a uma unica parte do projeto.

1. Estrutura inicial

Criem a estrutura do projeto e combinem antes as decisoes basicas de representacao. A ideia e que os dois possam trabalhar em arquivos diferentes e integrar com frequencia.

```
a-maze-ing/
- a_maze_ing.py
- config.txt
- config_parser.py
- maze_generator.py
- maze_solver.py
- display.py
- output.py
```

Padrao combinado:

```
maze[y][x]

NORTH = 1
EAST = 2
SOUTH = 4
WEST = 8
```

2. Primeiro ciclo - comecar agora

Pessoa 1	**Pessoa 2**
Trabalha em **config_parser.py**	Trabalha em **maze_generator.py**
Ler linhas KEY=VALUE	Criar a classe MazeGenerator
Converter WIDTH e HEIGHT	Criar a matriz maze[y][x]
Converter ENTRY e EXIT	Inicializar cada celula com 15
Converter PERFECT para bool	Criar funcoes basicas de acesso ao grid
Fazer validacoes basicas do config	Testar a criacao de um grid 5x5

Exemplo de config.txt para os testes iniciais:

```
WIDTH=5
HEIGHT=5
ENTRY=0,0
EXIT=4,4
OUTPUT_FILE=maze.txt
PERFECT=True
```

A Pessoa 2 nao precisa esperar o parser ficar pronto. Ela pode instanciar diretamente MazeGenerator(width=5, height=5) enquanto desenvolve a estrutura do grid.

3. Integracao do primeiro ciclo

Quando os dois terminarem suas tarefas, juntem as partes e testem a integracao:

```
config = read_config("config.txt")

generator = MazeGenerator(
width=config["width"],
height=config["height"]
)
```

Se funcionar, facam um commit pequeno e claro, por exemplo:

```
git commit -m "add config parser and maze structure"
```

4. Segundo ciclo - trocar as areas

Pessoa 1	**Pessoa 2**
Agora entra em **maze_generator.py**	Agora entra em **maze_solver.py**
Criar remove_wall()	Criar BFS
Trabalhar com vizinhos	Guardar celulas visitadas
Validar coordenadas	Encontrar um caminho entre entrada e saida
Comecar o DFS / Recursive Backtracker	Produzir a sequencia N/E/S/W

A ideia e que quem estava no parser passe a mexer no gerador, e quem iniciou o gerador passe a mexer no solver. Assim os dois realmente codam partes importantes do projeto.

5. Terceiro ciclo

Pessoa 1	**Pessoa 2**
Trabalha em **display.py**	Volta ao **maze_generator.py**
Criar visualizacao ASCII	Finalizar DFS
Desenhar paredes	Adicionar aleatoriedade
Mostrar ENTRY e EXIT	Adicionar seed
Depois mostrar o caminho	Fechar PERFECT=True

6. Quarto ciclo

Pessoa 1	**Pessoa 2**
Trabalha em **output.py**	Aprimora **display.py**
Converter celulas para hexadecimal	Adicionar comandos de usuario
Escrever maze.txt	Mostrar/esconder o caminho
Gravar ENTRY e EXIT	Regenerar o labirinto
Gravar shortest path	Trocar cores das paredes

7. Quinto ciclo - PERFECT=False

Essa e uma das partes mais delicadas. Em vez de uma pessoa ficar com tudo, dividam as subpartes e depois troquem para revisar e modificar o codigo um do outro.

Pessoa 1	Pessoa 2
Detectar dead-ends	Garantir conectividade total
Abrir paredes adicionais	Garantir centro aberto
Criar loops	Garantir os quatro cantos abertos
Reduzir becos sem saida	Verificar multiplas rotas independentes

Depois, invertam: Pessoa 1 revisa e altera a parte da Pessoa 2, e Pessoa 2 revisa e altera a parte da Pessoa 1. O objetivo e os dois realmente terem programado e entendido essa etapa.

8. Organizacao no Git

Usem duas branches de desenvolvimento para evitar que os dois mexam no mesmo arquivo ao mesmo tempo:

```
main
- dev-pessoa1
- dev-pessoa2
```

Exemplo:

```
Pessoa 1:
git switch -c dev-pessoa1

Pessoa 2:
git switch -c dev-pessoa2
```

Regra pratica: nao trabalhem simultaneamente no mesmo arquivo. Se um estiver em maze_generator.py, o outro fica em maze_solver.py, display.py, output.py ou outra parte independente.

9. Ritmo de trabalho recomendado

Trabalhem em ciclos curtos:

• Escolher duas tarefas pequenas e independentes.

• Cada pessoa trabalha em sua branch.

• Executar testes locais.

• Fazer commits pequenos.

• Integrar as duas partes.

• Testar o projeto completo.

• Trocar de area no ciclo seguinte.

10. Ordem completa sugerida

1. Config parser + estrutura do grid

2. remove_wall() + BFS

3. DFS/Recursive Backtracker + ASCII

4. PERFECT=True + shortest path

5. Hexadecimal + arquivo de output

6. Seed + regeneracao + cores

7. PERFECT=False

8. Pattern 42

9. Validacoes finais

10. Package mazegen-*

11. Makefile, flake8 e mypy

12. README e LICENSE

13. Testes finais e preparacao para avaliacao

11. Ponto de partida de hoje

Pessoa 1: criar config.txt + config_parser.py e transformar o arquivo em dados Python.

Pessoa 2: criar maze_generator.py + MazeGenerator e gerar uma matriz 5x5 completamente fechada.

Quando essas duas partes estiverem funcionando, integrem e passem para remove_wall() + BFS. Esse e o primeiro checkpoint real do projeto.